Lab 1 · wariant c) Y = (4 – X) mod 16

UCiSW 1 · Vivado + AUP-ZU3 · wszystko do zadań Z1–Z4, krok po kroku

Zad.	Co trzeba zrobić	Gdzie w tym PDF
Z1	Równania Y(0)…Y(3) z map Karnaugha, kod VHDL, symulacja behawioralna X = 0…15 (co 100 ns)	teoria + część Z1
Z2	Płyta: X → PL_USER_SW(3:0), Y → PL_USER_LED(3:0). **Dodatkowo:** symulacja czasowa przełączania bitów Y dla zmiany **Y: 8 → 7**	część Z2
Z3	Diagram blokowy: Z1 jako moduł + X i Y na OLED_1x32	część Z3
Z4	To samo działanie (4 – X) zrobione sumatorem/subtraktorem IP z biblioteki Vivado, wynik też na OLED	część Z4

Pliki w folderze `Lab1_c/`

Plik	Do czego	Zadanie
`Z1_c.vhd`	Układ (encja `Z1`, porty X, Y) — równania z map Karnaugha	Z1–Z4
`Z1_tb.vhd`	Testbench: X = 0…15 co 100 ns + automatyczne sprawdzanie wyników	Z1, Z2
`Z2.xdc`	Piny: X ← przełączniki SW3…0, Y → diody LED3…0	Z2
`Z3_Z4.xdc`	Piny dla diagramu: PL_USER_SW, OLED_SDA/SCL, NACK (LED7)	Z3, Z4
`OLED_1x32.vhd`, `ZU3.xdc`, `ZU3_Exp.xdc`	Pliki od prowadzącego (kopie z folderu `pliki/`)	Z3, Z4

Plan na zajęcia

Projekt **Z1** w Vivado służy do zadań Z1 i Z2 (najpierw symulacja, potem dokładasz XDC i wgrywasz na płytę). Projekt **Z3** to nowy projekt z diagramem blokowym; w Z4 tylko **dokładasz** klocki do tego samego diagramu.

Jak czytać instrukcję

Szary prostokąt = przycisk/menu w Vivado · `Ctrl`+`S` = skrót · **PPM** = prawy przycisk myszy · **2×klik** = podwójne kliknięcie · ramka **✔ SPRAWDŹ** = co masz zobaczyć.

Teoria: tablica prawdy i równania

X	X bin	Y	Y bin
0	`0000`	4	`0100`
1	`0001`	3	`0011`
2	`0010`	2	`0010`
3	`0011`	1	`0001`
4	`0100`	0	`0000`
5	`0101`	15	`1111`
6	`0110`	14	`1110`
7	`0111`	13	`1101`
8	`1000`	12	`1100`
9	`1001`	11	`1011`
10	`1010`	10	`1010`
11	`1011`	9	`1001`
12	**`1100`**	**8**	**`1000`**
13	**`1101`**	**7**	**`0111`**
14	`1110`	6	`0110`
15	`1111`	5	`0101`

Pogrubione wiersze: zmiana **Y: 8** → **7** z zadania Z2 (X: 12 → 13).

Szybki sposób na tablicę

4 – X: dla X = 0…4 wychodzi 4, 3, 2, 1, 0; dla X ≥ 5 wynik jest ujemny, więc dodajesz 16: 5 → 15, 6 → 14, … 15 → 5.

Ciekawostki do powiedzenia

- **Y(0) = X(0)** — odejmowanie od liczby parzystej (4) nie zmienia parzystości.
- **$Y(1) = x_1 \oplus x_0$** — na mapie „szachownica” kolumn, nic się nie skleja.
- W FPGA każde Y(i) zależy od ≤ 4 wejść → jedna tablica **LUT6**; Vivado i tak sam optymalizuje logikę.

Równania (do VHDL)

$Y(0) = x_0$

$Y(1) = x_1\overline{x_0} + \overline{x_1}x_0$

$Y(2) = x_2x_1 + x_2x_0 + \overline{x_2}\,\overline{x_1}\,\overline{x_0}$

$Y(3) = x_3\overline{x_2} + x_3\overline{x_1}\,\overline{x_0} + \overline{x_3}x_2x_1 + \overline{x_3}x_2x_0$

Wszystkie równania sprawdzone dla 16 wartości X (skrypt + symulacja testbencha).

Mapy Karnaugha z grupami

Wiersze x_3x_2, kolumny x_1x_0 (kod Graya). Szara liczba w rogu = X. Kolorowe paski = grupy; kolor paska = kolor składnika w równaniu. Skrajne kolumny i wiersze są sąsiednie.

Y(3)

$x_3x_2 \backslash x_1x_0$	00	01	11	10
00	0 (0)	0 (1)	0 (3)	0 (2)
01	0 (4)	1 (5)	1 (7)	1 (6)
11	1 (12)	0 (13)	0 (15)	0 (14)
10	1 (8)	1 (9)	1 (11)	1 (10)

$Y(3) = x_3\bar{x_2} + x_3\bar{x_1}\bar{x_0} + \bar{x_3}x_2x_1 + \bar{x_3}x_2x_0$

$= x_3 \oplus (x_2(x_1 + x_0))$

Y(2)

$x_3x_2 \backslash x_1x_0$	00	01	11	10
00	1 (0)	0 (1)	0 (3)	0 (2)
01	0 (4)	1 (5)	1 (7)	1 (6)
11	0 (12)	1 (13)	1 (15)	1 (14)
10	1 (8)	0 (9)	0 (11)	0 (10)

$Y(2) = x_2x_1 + x_2x_0 + \bar{x_2}\bar{x_1}\bar{x_0}$

$= x_2 \odot (x_1 + x_0)$ (XNOR)

Y(1)

$x_3x_2 \backslash x_1x_0$	00	01	11	10
00	0 (0)	1 (1)	0 (3)	1 (2)
01	0 (4)	1 (5)	0 (7)	1 (6)
11	0 (12)	1 (13)	0 (15)	1 (14)
10	0 (8)	1 (9)	0 (11)	1 (10)

$Y(1) = x_1\bar{x_0} + \bar{x_1}x_0$

$= x_1 \oplus x_0$

Y(0)

$x_3x_2 \backslash x_1x_0$	00	01	11	10
00	0 (0)	1 (1)	1 (3)	0 (2)
01	0 (4)	1 (5)	1 (7)	0 (6)
11	0 (12)	1 (13)	1 (15)	0 (14)
10	0 (8)	1 (9)	1 (11)	0 (10)

$Y(0) = x_0$

$= Y(0) = X(0)$ — zwykły przewód

Grupy dla Y(3)

- $x_3\bar{x_2}$: cały wiersz 10 (X = 8…11)
- $x_3\bar{x_1}\bar{x_0}$: kolumna 00, wiersze 11 i 10 (X = 12, 8)
- $\bar{x_3}x_2x_1$: wiersz 01, kolumny 11 i 10 (X = 7, 6)
- $\bar{x_3}x_2x_0$: wiersz 01, kolumny 01 i 11 (X = 5, 7)

Grupy dla Y(2)

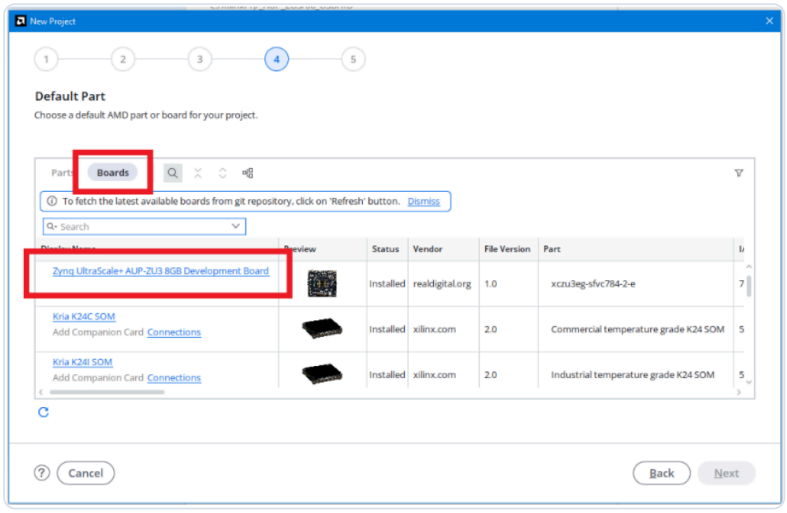
- x_2x_1: wiersze 01 i 11, kolumny 11 i 10 (X = 6, 7, 14, 15)
- x_2x_0: wiersze 01 i 11, kolumny 01 i 11 (X = 5, 7, 13, 15)
- $\bar{x_2}\bar{x_1}\bar{x_0}$: kolumna 00, wiersze 00 i 10 (X = 0, 8 — skrajne wiersze są sąsiednie)

Z1 · projekt, kod, symulacja behawioralna

Cel: działający kod VHDL i wykres X = 0…15.

1 Nowy projekt

1. Uruchom Vivado → **Create Project** → **Next**.
2. Project name: `Z1`, location: `C:/AMD_Proj`, ☑ Create project subdirectory → **Next**. **Bez polskich znaków i spacji!**
3. ◉ **RTL Project**, ☑ **Do not specify sources at this time** → **Next**.
4. Zakładka **Boards** → szukaj `AUP` → kliknij **Zynq UltraScale+ AUP-ZU3 8GB Development Board** → **Next** → **Finish**.



Boards → AUP-ZU3 8GB

✔ **SPRAWDŹ** W Project Summary: Target language = **VHDL** (jeśli Verilog — kliknij i zmień).

2 Plik układu Z1

1. **Add Sources** (`Alt`+`A`) → ◉ **Add or create design sources** → **Next**.
2. **Jeśli masz plik:** **Add Files** → `Z1_c.vhd` → ☑ **Copy sources into project** → **Finish**. Gotowe — przejdź do kroku 3.
3. **Jeśli piszesz ręcznie:** **Create File** → VHDL, nazwa `Z1` → **OK** → **Finish** → w oknie Define Module wpisz porty:

Port Name	Direction	Bus	MSB	LSB
X	in	☑	3	0
Y	out	☑	3	0

→ **OK** → 2×klik na **Z1** w Sources → między `begin` a `end Behavioral;` wpisz:

```
Y(0) <= X(0);
Y(1) <= (X(1) and not X(0)) or (not X(1) and X(0));
Y(2) <= (X(2) and X(1)) or (X(2) and X(0))
     or (not X(2) and not X(1) and not X(0));
Y(3) <= (X(3) and not X(2)) or (X(3) and not X(1) and not X(0))
     or (not X(3) and X(2) and X(1)) or (not X(3) and X(2) and X(0));
```

→ Ctrl+S.

✔ SPRAWDŹ Na prawym brzegu edytora zielony kwadracik (brak błędów). W Design Sources jest **Z1**.

3 Testbench

1. Add Sources → ◉ **Add or create simulation sources** → Next.
2. Masz plik: Add Files → `Z1_tb.vhd` → ☑ Copy → Finish.
   Ręcznie: Create File → VHDL, `Z1_tb` → Define Module **bez portów** → OK → Yes → otwórz `Z1_tb`, Ctrl+A, wklej kod poniżej, Ctrl+S.

```
-- Testbench Z1 c): X = 0..15, co 100 ns. Zmiana Y: 8 -> 7 nastepuje w 1300 ns (X: 12 ->
13).
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Z1_tb is
end Z1_tb;

architecture Behavioral of Z1_tb is
    component Z1
        Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
               Y : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

    signal X : STD_LOGIC_VECTOR (3 downto 0);
    signal Y : STD_LOGIC_VECTOR (3 downto 0);
begin
    UUT: Z1 port map ( X => X, Y => Y );

    X <= "0000",
         "0001" after  100 ns, "0010" after  200 ns, "0011" after  300 ns,
         "0100" after  400 ns, "0101" after  500 ns, "0110" after  600 ns,
         "0111" after  700 ns, "1000" after  800 ns, "1001" after  900 ns,
         "1010" after 1000 ns, "1011" after 1100 ns, "1100" after 1200 ns,
         "1101" after 1300 ns, "1110" after 1400 ns, "1111" after 1500 ns;

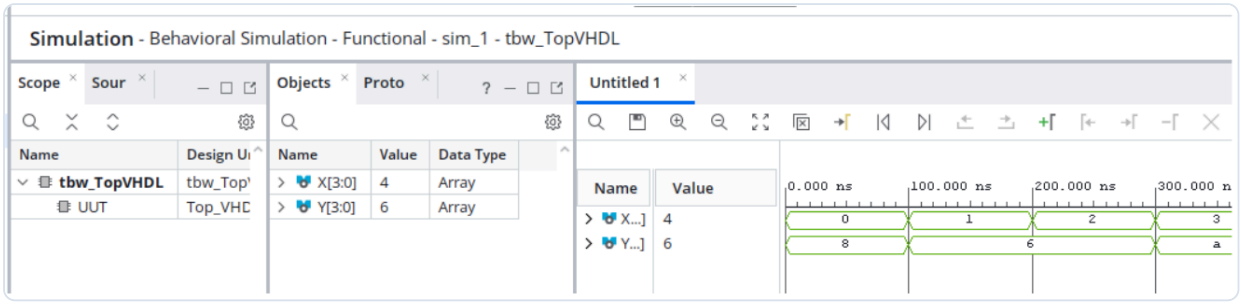
    -- Samosprawdzenie (opcjonalne): w polowie kazdego kroku Y musi byc (4 - X) mod 16
    Check: process
    begin
        for i in 0 to 15 loop
            wait for 50 ns;
            assert to_integer(unsigned(Y)) = (4 - i) mod 16
                report "Blad dla X = " & integer'image(i) severity error;
            wait for 50 ns;
        end loop;
        report "Koniec testu";
        wait;
    end process;
end Behavioral;
```

Proces `Check` jest opcjonalny (sprawdza wyniki za Ciebie). Wymagane przez polecenie jest tylko przypisanie `X <= "0000", "0001" after 100 ns, …`.

✔ SPRAWDŹ Simulation Sources → sim_1: **Z1_tb** pogrubiony (top), pod nim **UUT : Z1**. Jeśli nie: PPM na Z1_tb → Set as Top.

4 Symulacja behawioralna

1. Flow Navigator → SIMULATION → **Run Simulation** → **Run Behavioral Simulation**.
2. Domyślnie symulacja trwa 1000 ns, a test 1600 ns: w zakładce **Tcl Console** (na dole) wpisz `run 600 ns` + **Enter**.
3. **Zoom Fit** (⛶) nad wykresem.
4. PPM na **X** → **Radix** → **Unsigned Decimal**. To samo dla **Y**.



Okno symulacji z wykresem X i Y

✔ **SPRAWDŹ** Pod X = 0, 1, 2, 3, 4, 5 … 15 widać Y = 4, 3, 2, 1, 0, 15 … 5. W Tcl Console: „Koniec testu”, **brak** „Blad dla X”.

⚠ **Pokaż prowadzącemu**
mapy Karnaugha + ten wykres = zaliczone Z1.

Z2 · implementacja na płycie + symulacja czasowa

Ten sam projekt Z1 — dokładasz piny i wgrywasz.

1 Plik XDC (piny)

Wróć do edycji: **PROJECT MANAGER** w Flow Navigatorze.

1. **Add Sources** → ◉ **Add or create constraints** → **Next** → **Add Files** → `Z2.xdc` → ☑ **Copy constraints files into project** → **Finish**.
2. **Albo z szablonu** `ZU3.xdc`: odkomentuj (usuń `#`) linie `PL_USER_SW[0..3]` + IOSTANDARD i `PL_USER_LED[0..3]` + IOSTANDARD, zmień nazwy na `X` i `Y`. Efekt:

```
## Z2: X(3:0) <- PL_USER_SW(3:0), Y(3:0) -> PL_USER_LED(3:0)  (linie z ZU3.xdc)

set_property PACKAGE_PIN AB1 [get_ports {X[0]}]
set_property PACKAGE_PIN AF1 [get_ports {X[1]}]
set_property PACKAGE_PIN AE3 [get_ports {X[2]}]
set_property PACKAGE_PIN AC2 [get_ports {X[3]}]
set_property IOSTANDARD LVCMOS12 [get_ports X*]

set_property PACKAGE_PIN AF5 [get_ports {Y[0]}]
set_property PACKAGE_PIN AE7 [get_ports {Y[1]}]
set_property PACKAGE_PIN AH2 [get_ports {Y[2]}]
set_property PACKAGE_PIN AE5 [get_ports {Y[3]}]
set_property IOSTANDARD LVCMOS12 [get_ports Y*]
```

> ⚠
> Nie zapomnij linii `IOSTANDARD`. Nazwy `X` / `Y` muszą być identyczne jak w VHDL.

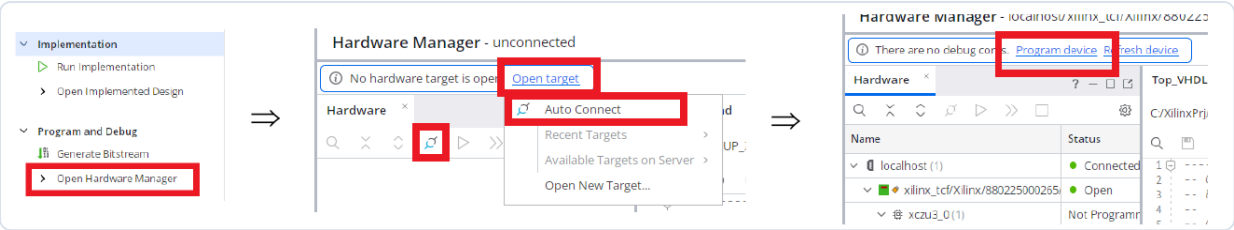
2 Bitstream

1. Flow Navigator → PROGRAM AND DEBUG → **Generate Bitstream** → **Yes** → **OK**.
2. Czekaj kilka minut (postęp w prawym górnym rogu).
3. Okno „Bitstream Generation Completed” → ◉ **Open Hardware Manager** → **OK**.

✔ SPRAWDŹ Prawy górny róg: **write_bitstream Complete**. Błędy UCIO-1 / NSTD-1 = coś nie tak w XDC.

3 Wgranie i test na płycie

1. Płyta podłączona USB i włączona.
2. Hardware Manager: **Open target** → **Auto Connect** → **Program device** → plik `Z1.runs/impl_1/Z1.bit` → **Program**.



Open target → Auto Connect → Program device

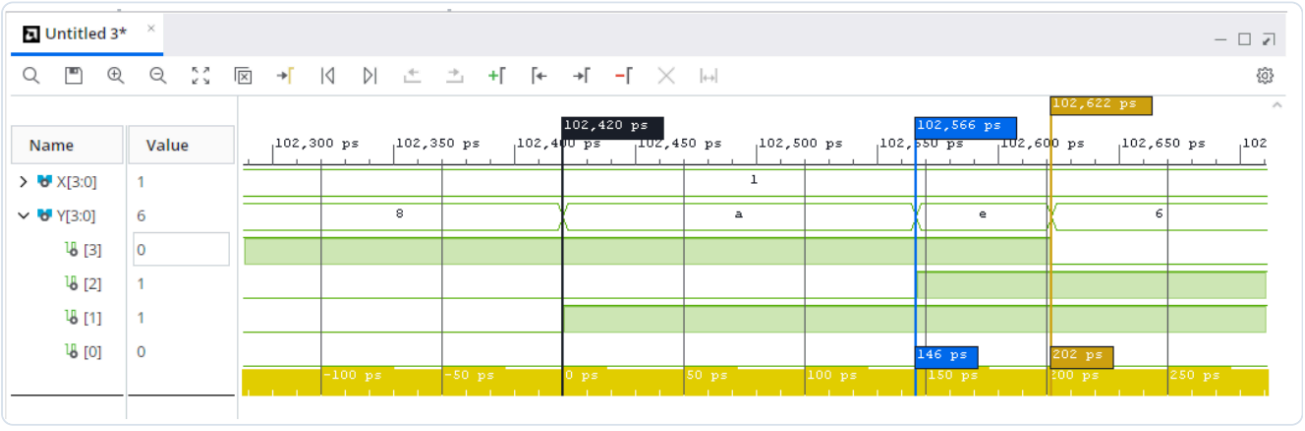
SW3 SW2 SW1 SW0 (X)	LED3 LED2 LED1 LED0 (Y)
0000 (0)	0100 (4)
0100 (4)	0000 (0)
0101 (5)	1111 (15)
1100 (12)	1000 (8)
1101 (13)	0111 (7)
1111 (15)	0101 (5)

✔ **SPRAWDŹ** Diody zgadzają się z tabelą (1 = świeci).

4 DODATKOWO: symulacja czasowa dla Y: 8 → 7

Y = 8 dla X = 12, a Y = 7 dla X = 13. W testbenchu X zmienia się 12 → 13 w chwili **1300 ns**.

1. Flow Navigator → SIMULATION → **Run Simulation** → **Run Post-Implementation Timing Simulation**.
2. Tcl Console: `run 500 ns` (żeby dojść za 1300 ns).
3. Powiększ okolice **1300 ns** (**Zoom In** lub `Ctrl`+kółko).
4. Kliknij ▸ przy **Y** — rozwiną się bity Y[3], Y[2], Y[1], Y[0].
5. Pomiar: kursor na zbocze X → **Add Marker** → zaznacz bit Y → **Next Transition**. Zrób to dla każdego bitu.



Przykład z instrukcji: rozwinięte bity Y i markery z czasami

Analiza: co się dzieje przy Y: 8 → 7

	X	Y	Y[3]	Y[2]	Y[1]	Y[0]
przed (do 1300 ns)	1100 (12)	1000 (8)	1	0	0	0
po	1101 (13)	0111 (7)	0 ↓	1 ↑	1 ↑	1 ↑

Najważniejsze: zmienia się tylko 1 bit wejścia, ale WSZYSTKIE 4 bity wyjścia

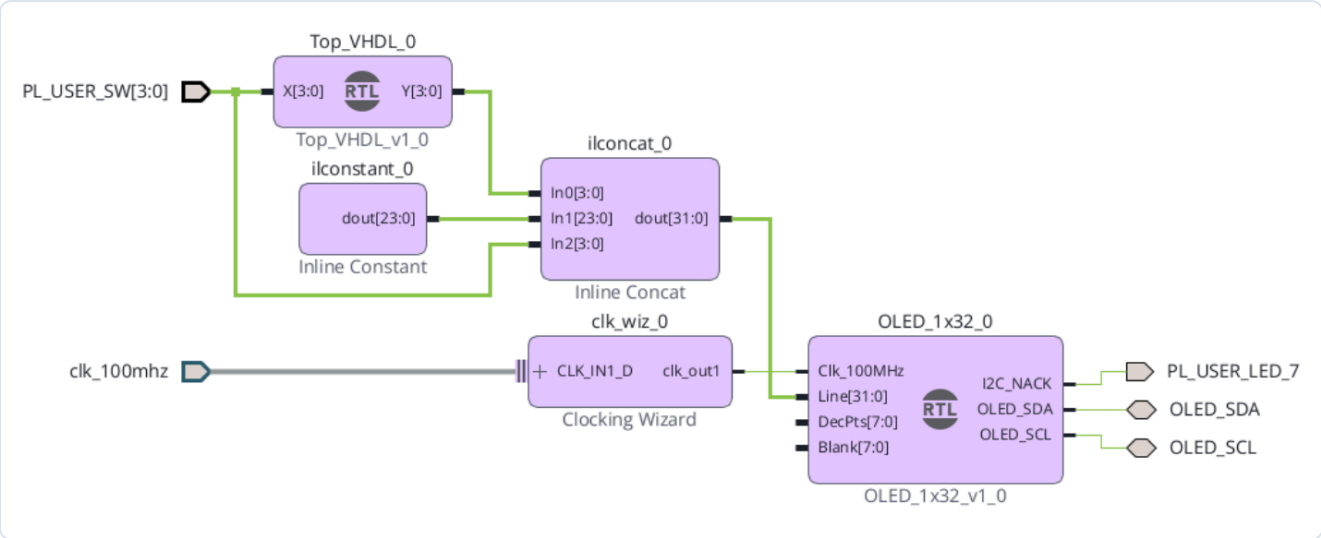
- Na X zmienia się tylko X(0) (0 → 1), a na Y przełączają się **wszystkie 4 bity** — to najgorszy możliwy przypadek.
- Każdy bit Y ma własną ścieżkę (inna LUT, inne połączenia, inny pin), więc bity zmieniają się w **nieco różnych chwilach** (różnice rzędu ułamków ns).
- Przez ten krótki czas na Y pojawiają się **fałszywe wartości pośrednie** (hazard / glitch), np. `1111` (15), jeśli Y[3] spadnie ostatni, albo `0000` (0), jeśli spadnie pierwszy, a pozostałe wzrosną później. Możliwe są wszystkie 14 stanów „po drodze”.
- **Y[0] = X[0]** — to tylko przewód (bez logiki), więc zwykle zmienia się najszybciej; Y[1]…Y[3] przechodzą przez LUT.
- Całe opóźnienie (od zmiany X do ustalenia Y) to kilka ns: bufor wejściowy → połączenia → LUT → bufor wyjściowy.

Wniosek do powiedzenia

Wyjście układu kombinacyjnego jest poprawne dopiero, gdy ustalą się **wszystkie** bity. Dlatego w układach synchronicznych wynik próbkuje się zegarem, a okres zegara musi być dłuższy niż najdłuższa ścieżka.

Z3 · diagram blokowy: Z1 + OLED

Nowy projekt. Moduł Z1 jako klocek, X i Y na wyświetlaczu.



Wzór z instrukcji prowadzącego (u Ciebie Top_VHDL_0 = Z1_0). Wynik na OLED: X po lewej, Y po prawej.

1 Nowy projekt Z3 i pliki

1. Nowy projekt jak w Z1 (krok 1), nazwa `Z3`.
2. **Add Sources** → design sources → **Add Files**: `Z1_c.vhd` i `OLED_1x32.vhd` → ☑ Copy → **Finish**.
3. **Add Sources** → constraints → **Add Files**: `Z3_Z4.xdc` → ☑ Copy → **Finish**.

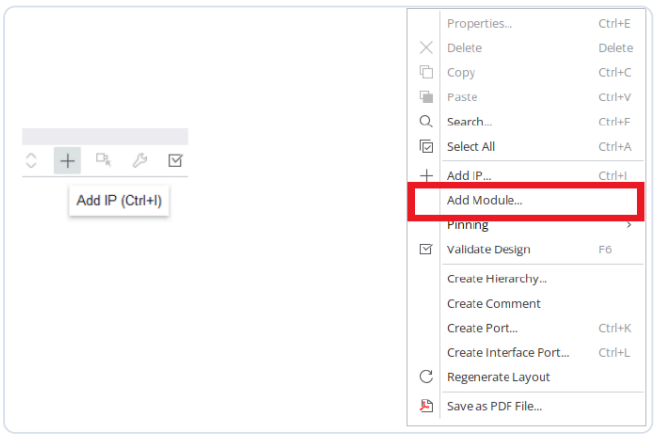
✔ SPRAWDŹ Design Sources: **Z1** i **OLED_1x32**. Constraints: **Z3_Z4.xdc**.

2 Pusty diagram

Flow Navigator → IP INTEGRATOR → **Create Block Design** → nazwa `Z3_BD` → **OK**.

3 Klocki Z1 i OLED

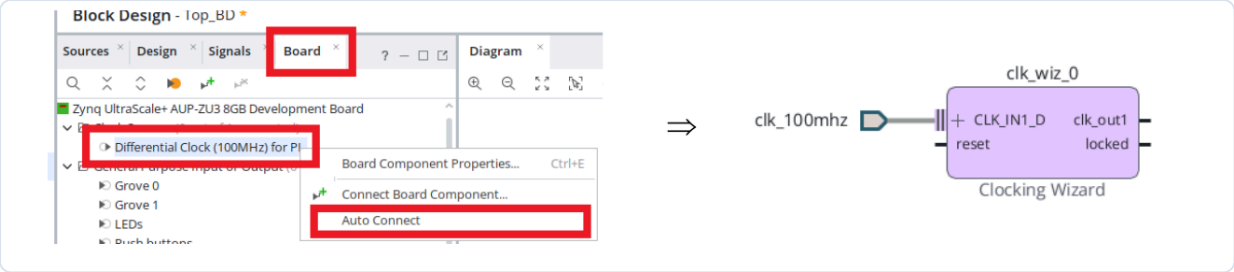
PPM na planszy → **Add Module...** → **Z1** → **OK**. Powtórz dla **OLED_1x32**.



PPM na planszy → Add Module

4 Zegar (Clocking Wizard)

1. Lewy panel → zakładka **Board** → **Differential Clock (100MHz) for PL** → PPM → **Auto Connect**.
2. 2×klik na **clk_wiz_0** → zakładka **Output Clocks** → na dole odznacz □ **reset** i □ **locked** → **OK** (nie znajdziesz — zostaw).



Board → Differential Clock → Auto Connect

> ⚠
> W XDC **nie** opisujesz CLK_100MHz — wizard robi to sam (dlatego nie ma tego w `Z3_Z4.xdc`).

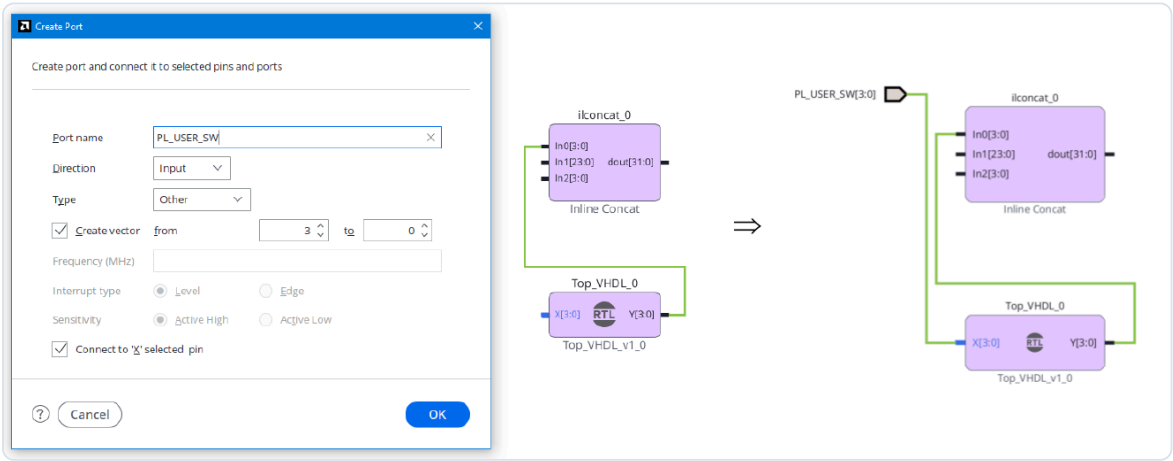
5 Stałe i Concat

1. `+` Add IP (`Ctrl`+`I`) → `constant` → **Inline Constant**. 2×klik: Const Width = `24`, Const Val = `0`.
2. `+` → `concat` → **Inline Concat**. 2×klik: Number of Ports = `3`, In0 Width = `4`, In1 Width = `24`, In2 Width = `4`.
3. Jeszcze jedna stała do ukrycia zer: **Inline Constant**, Const Width = `8`, Const Val = `126` (= 0111 1110: wygaś 6 środkowych cyfr).

Nazwy pól mogą się lekko różnić w zależności od wersji Vivado.

6 Port przełączników

PPM na nóżce **X[3:0]** klocka Z1_0 → **Create Port...** (`Ctrl`+`K`): Port name `PL_USER_SW`, Direction **Input**, ☑ Create vector `3` to `0`, ☑ Connect to selected pin → **OK**.



Create Port: PL_USER_SW[3:0]

7 Połączenia (przeciągnij myszką od nóżki do nóżki)

#	Od	Do	Po co
1	port **PL_USER_SW[3:0]**	Concat **In2[3:0]**	X na lewej cyfrze
2	Z1_0 **Y[3:0]**	Concat **In0[3:0]**	Y na prawej cyfrze
3	stała 24-bit **dout[23:0]**	Concat **In1[23:0]**	zera w środku
4	Concat **dout[31:0]**	OLED **Line[31:0]**	co wyświetlić
5	stała 8-bit **dout[7:0]**	OLED **Blank[7:0]**	ukryj zera
6	clk_wiz_0 **clk_out1**	OLED **Clk_100MHz**	zegar

Pomyłka: PPM na nóżce → Disconnect Pin. DecPts zostaw niepodłączone.

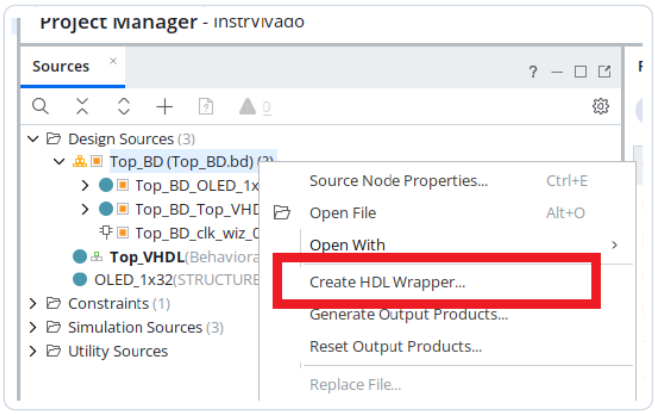
8 Wyjścia OLED i NACK

1. PPM na nóżce **OLED_SDA** → **Make External** (`Ctrl`+`T`). To samo: **OLED_SCL**, **I2C_NACK**.
2. Zmień nazwy portów (kliknij port → okienko **External Port Properties** → pole Name): `OLED_SDA_0` → `OLED_SDA`, `OLED_SCL_0` → `OLED_SCL`, `I2C_NACK_0` → `NACK`.

> ⚠
> Nazwy muszą być takie jak w `Z3_Z4.xdc`: PL_USER_SW, OLED_SDA, OLED_SCL, NACK.

9 Porządek, walidacja, wrapper

1. **Regenerate Layout** (⟳), potem **Validate Design** (`F6`) → `Ctrl`+`S`.
2. Sources → PPM na **Z3_BD** (.bd) → **Create HDL Wrapper...** → ◉ **Let Vivado manage wrapper and auto-update** → **OK**.



Create HDL Wrapper

✔ **SPRAWDŹ** Validate: „successful” (żółte ostrzeżenia o DecPts są OK). Top w Design Sources: **Z3_BD_wrapper**.

10 Bitstream, wgranie, test

Generate Bitstream → **Yes** → **OK** → Hardware Manager → **Program device** (`Z3_BD_wrapper.bit`).

✔ **SPRAWDŹ** OLED: lewa cyfra = X, prawa = Y, **szesnastkowo**. Np. SW = 1101 → `D 7` ; SW = 0101 → `5 F` . LED7 nie świeci.

Z4 · to samo działanie przez IP Adder/Subtracter

W tym samym diagramie Z3 dokładasz gotowy subtraktor z biblioteki Vivado: S = 4 – X.

Cel na wyświetlaczu

`X YS` — lewa cyfra X, przedostatnia Y z Twojego układu Z1, ostatnia S z subtraktora IP. **Dwie ostatnie cyfry muszą być zawsze równe** — to dowód, że Twoje równania są poprawne.

Wejście Concat	Szerokość	Podłącz	Cyfra na OLED
In0	4	S z subtraktora	0 (prawa)
In1	4	Y z Z1_0	1
In2	20	stała 0 (20 bitów)	2–6 (ukryte)
In3	4	X (PL_USER_SW)	7 (lewa)

1 Otwórz diagram Z3

Ten sam projekt Z3. Flow Navigator → IP INTEGRATOR → **Open Block Design** (albo 2×klik na **Z3_BD** w Sources).

2 Dodaj subtraktor IP

1. `+` Add IP → wpisz `adder` → 2×klik **Adder/Subtracter**.
2. 2×klik na nowy klocek (**c_addsub_0**). Zakładka **Basic**:
   - Implement using: **Fabric**
   - Operation: **Subtract** (wynik = A – B)
   - A: Input Type **Unsigned**, Width `4` · B: Input Type **Unsigned**, Width `4`
   - Output Width: `4` (wynik mod 16)
   - Latency Configuration: **Manual**, Latency: `0` → układ czysto kombinacyjny, bez zegara

→ **OK**.

Jeśli Output Width nie da się ustawić na 4 (np. jest 5) — zostaw; wtedy dodaj klocek **Inline Slice** (Din Width = 5, Din From = 3, Din Down To = 0) między S a Concat.

✔ SPRAWDŹ Klocek ma wejścia **A[3:0]**, **B[3:0]** i wyjście **S[3:0]**, bez wejścia CLK.

3 Stała 4 na wejście A

`+` → **Inline Constant** → Const Width = `4`, Const Val = `4`. Połącz jej **dout[3:0]** z **A[3:0]** subtraktora. Połącz port **PL_USER_SW[3:0]** z **B[3:0]**. Teraz S = 4 – X.

4 Przebuduj Concat i stałe

1. Zaznacz stary klocek Concat → klawisz `Delete` (jego połączenia znikną).
2. Dodaj nowy `+` → **Inline Concat**: Number of Ports = `4`, In0 = `4`, In1 = `4`, In2 = `20`, In3 = `4`.
3. 2×klik na stałą 24-bitową → zmień Const Width na `20`.

4. 2×klik na stałą 8-bitową (Blank) → Const Val = `124` (= 0111 1100: wygaś cyfry 2–6).
5. Połącz według tabeli z początku Z4: S → In0, Y → In1, stała 20 → In2, PL_USER_SW → In3, Concat dout[31:0] → OLED Line[31:0].

5 Walidacja i zapis

Regenerate Layout → **Validate Design** (**F6**) → **Ctrl**+**S**. Wrapper zaktualizuje się sam (wybrałeś „auto-update”).

6 Bitstream, wgranie, test

Generate Bitstream → **Program device** (`Z3_BD_wrapper.bit`). Pierwsza synteza z nowym IP trwa dłużej.

SW (X)	OLED
0000 (0)	`0 44`
0101 (5)	`5 FF`
1100 (12)	`C 88`
1101 (13)	`D 77`

✔ **SPRAWDŹ** Dwie ostatnie cyfry zawsze równe = Z1 i subtraktor IP liczą to samo.

Coś nie działa

Objaw	Co zrobić
Brak płyty AUP-ZU3 na liście	Zakładka **Boards**, szukaj „AUP”. Dalej brak → prowadzący.
Czerwona kreska w kodzie	Najedź na nią — komunikat błędu. Zwykle brak `;` albo `A and B or C` bez nawiasów.
Na wykresie tylko X = 0…9	Tcl Console: `run 600 ns` .
Y czerwone / „U”	Hierarchia: Z1_tb → UUT : Z1; Set as Top na Z1_tb.
Bitstream: `UCIO-1` / `NSTD-1`	Brak PACKAGE_PIN lub IOSTANDARD w XDC.
Critical Warning „No ports matched …”	Nazwa w XDC ≠ nazwa portu (np. `OLED_SDA_0`). Zmień nazwę portu w diagramie.
„No hardware target”	Kabel USB, zasilanie, ponownie Open target → Auto Connect.
Płyta działa po staremu	Po każdej zmianie: Generate Bitstream + Program device. Sprawdź datę pliku .bit.
OLED pusty / świeci LED7	Płyta rozszerzająca wpięta? Zegar do Clk_100MHz? Linie OLED w XDC z LVCMOS33?
Subtraktor ma wejście CLK	Latency ≠ 0 — ustaw Manual, Latency = 0.
Ostatnie dwie cyfry różne (Z4)	Błąd w Z1 (porównaj z symulacją) albo zamienione A i B w subtraktorze (A = stała 4, B = X).
Wszystko się sypie	Tcl Console: `reset_project` , potem Generate Bitstream.

> **Złota zasada**
> Po zmianie: `Ctrl`+`S` → symulacja: Relaunch → płyta: Generate Bitstream → Program device.